

C++ Introduction

C++ is a cross-platform language that can be used to create high-performance applications.

C++ was developed by Bjarne Stroustrup, as an extension to the C language.

C++ gives programmers a high level of control over system resources and memory.

C++ is one of the world's most popular programming languages.

C++ can be found in today's operating systems, Graphical User Interfaces, and embedded systems.

C++ is an object-oriented programming language which gives a clear structure to programs and allows code to be reused, lowering development costs.

C++ is portable and can be used to develop applications that can be adapted to multiple platforms.

As C++ is close to C, C# and Java, it makes it easy for programmers to switch to C++ or vice versa.

Example 1

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World!";
    return 0;
}
```

Line 1: `#include <iostream>` is a **header file library** that lets us work with input and output objects, such as `cout` (used in line 5). Header files add functionality to C++ programs.

Line 2: `using namespace std` means that we can use names for objects and variables from the standard library.

Line 3: Another thing that always appear in a C++ program is `int main()`. This is called a **function**. Any code inside its curly brackets `{ }` will be executed.

Line 4: `cout` (pronounced "see-out") is an **object** used together with the *insertion operator* (`<<`) to output/print text. In our example, it will output "Hello World!".

Note: C++ is case-sensitive: "cout" and "Cout" has different meaning.

Note: Every C++ statement ends with a semicolon `;`.

Note: The body of `int main()` could also been written as:
`int main () { cout << "Hello World! "; return 0; }`

Remember: The compiler ignores white spaces. However, multiple lines makes the code more readable.

Omitting Namespace

You might see some C++ programs that runs without the standard namespace library. The `using namespace std` line can be omitted and replaced with the `std` keyword, followed by the `::` operator for some objects:

Example 2

```
#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

C++ Statements

A **computer program** is a list of "instructions" to be "executed" by a computer.

In a programming language, these programming instructions are called **statements**.

C++ Output (Print Text)

The `cout` object, together with the `<<` operator, is used to output values and print text.

You can add as many `cout` objects as you want. However, note that it does not insert a new line at the end of the output

New Lines

To insert a new line in your output, you can use the `\n` character:

Example without new line

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    cout << "Welome to AKCIT";
    return 0;
}
```

Example with new line

```
#include <iostream>
using namespace std;

int main() {
```

```
cout << "Hello World! \n";
cout << "Welcome to akcit";
return 0;
}
```

You can also use another `<<` operator and place the `\n` character after the text, like this:

Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!" << "\n";
    cout << " Welcome to akcit";
    return 0;
}
```

Another way to insert a new line, is with the `endl` manipulator:

Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!" << endl;
    cout << " Welcome to akcit";
    return 0;
}
```

Escape Sequence

`\t` - Creates a horizontal tab

`\\` - Inserts a backslash character (`\`)

`\"` - Inserts a double quote character

`\'` - Inserts a single quote character

Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World! \t";
    cout << "Welcome to akcit";
}
```

```
return 0;  
}
```

Example

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
  
    cout << "Welcome to \"AKCIT\".";   
  
    return 0;  
  
}
```

C++ Output (Print number)

unlike text, we don't put numbers inside double quotes

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
  
    cout << 4 + 3;  
  
    return 0;  
  
}
```

C++ Comments

Comments can be used to explain C++ code, and to make it more readable. It can also be used to prevent execution when testing alternative code. Comments can be singled-lined or multi-lined.

Single-line Comments

Single-line comments start with two forward slashes (//).

Any text between // and the end of the line is ignored by the compiler (will not be executed).

C++ Multi-line Comments

Multi-line comments start with /* and ends with */.

Any text between /* and */ will be ignored by the compiler:

C++ Variables

Variables are containers for storing data values.

Syntax

type variableName = value;

Example

```
int myNum = 5;           // Integer (whole number without decimals)
double myFloatNum = 5.99; // Floating point number (with decimals)
char myLetter = 'D';     // Character
string myText = "Hello"; // String (text)
bool myBoolean = true;   // Boolean (true or false)
```

Declare Many Variables

To declare more than one variable of the **same type**, use a comma-separated list:

```
int x = 5, y = 6, z = 50;
```

You can also assign the **same value** to multiple variables in one line:

```
int x, y, z;
x = y = z = 50;
```

C++ Identifiers

All C++ **variables** must be **identified** with **unique names**.

These unique names are called **identifiers**.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

The general rules for naming variables are:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (_)
- Names are case-sensitive (**myVar** and **myvar** are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (like C++ keywords, such as **int**) cannot be used as names

Constants

When you do not want others (or yourself) to change existing variable values, use the **const** keyword (this will declare the variable as "constant", which means **unchangeable and read-only**):

```
const int myNum = 15; // myNum will always be 15
myNum = 10; // error: assignment of read-only variable 'myNum'
```

When you declare a constant variable, it must be assigned with a value:

Example - `const int minutesPerHour = 60;`

Example - `const int minutesPerHour;`
`minutesPerHour = 60; // error , it will not Work`

C++ User Input

`cin` is a predefined variable that reads data from the keyboard with the extraction operator (`>>`)

Example

```
int x;
cout << "Type a number: "; // Type a number and press enter
cin >> x; // Get user input from the keyboard
cout << "Your number is: " << x; // Display the input value
```

C++ Data Types

Basic Data Types/ Built in data types

Data Type	Size
boolean	1 byte
char	1 byte
int	2 bytes
long int	4 bytes
short int	2 bytes
float	4 bytes
double	8 bytes
long double	10 bytes

Boolean Types

A boolean data type is declared with the `bool` keyword and can only take the values `true` or `false`.

When the value is returned, `true = 1` and `false = 0`.

example

```
bool isCodingFun = true;
bool isFishTasty = false;
cout << isCodingFun; // Outputs 1 (true)
cout << isFishTasty; // Outputs 0 (false)
```

Character Types

The `char` data type is used to store a **single** character. The character must be surrounded by single quotes, like 'A' or 'c':

if you are familiar with ASCII, you can use ASCII values to display certain characters:

Example

```
#include <iostream>

using namespace std;

int main () {

    char a = 65, b = 66, c = 67;

    cout << a;

    cout << b;

    cout << c;

    return 0;

}
```

C++ Strings

Strings are used for storing text/characters.

For example, "Hello World" is a string.

A **string** variable contains a collection of characters surrounded by double quotes:

String Concatenation

The **+** operator can be used between strings to add them together to make a new string. This is called **concatenation**:

```
string firstName = "John ";
string lastName = "Doe";
string fullName = firstName + lastName;
cout << fullName;
```

In the example above, we added a space after firstName to create a space between John and Doe on output. However, you could also add a space with quotes (" " or ' '):

Append

A string in C++ is actually an object, which contain functions that can perform certain operations on strings. For example, you can also concatenate strings with the **append()** function:

```
string firstName = "John ";
string lastName = "Doe";
```

```
string fullName = firstName.append(lastName);
cout << fullName;
```

C++ string Function

Function	Description
at()	Returns an indexed character from a string
length()	Returns the length of a string
size()	Alias of length(). Returns the length of a string
max_size()	Returns the maximum length of a string
empty()	Checks wheter a string is empty or not
append()	Appends a string (or a part of a string) to another string
substr()	Returns a part of a string from a start index (position) and length
find()	Returns the index (position) of the first occurrence of a string or character
rfind()	Returns the index (position) of the last occurrence of a string or character
replace()	Replaces a part of a string with another string
insert()	Inserts a string at a specified index (position)
erase()	Removes characters from a string
compare()	Compares two strings

C++ uses the **+** operator for both **addition** and **concatenation**.

Numbers are added. Strings are concatenated.

Access Strings

You can access the characters in a string by referring to its index number inside square brackets **[]**.

This example prints the **first character** in **myString**:

```
string myString = "Hello";
cout << myString[0];
// Outputs H
```

Change String Characters

To change the value of a specific character in a string, refer to the index number, and use single quotes:

```
string myString = "Hello";
myString[0] = 'J';
```

```
cout << myString;  
// Outputs Jello instead of Hello
```

User Input Strings

It is possible to use the extraction operator `>>` on `cin` to store a string entered by a user:

`cin` considers a space (whitespace, tabs, etc) as a terminating character, which means that it can only store a single word (even if you type many words):

```
string fullName;  
cout << "Type your full name: ";  
cin >> fullName;  
cout << "Your name is: " << fullName;
```

```
// Type your full name: John Doe  
// Your name is: John
```

From the example above, you would expect the program to print "John Doe", but it only prints "John".

That's why, when working with strings, we often use the `getline()` function to read a line of text. It takes `cin` as the first parameter, and the string variable as second:

```
string fullName;  
cout << "Type your full name: ";  
getline (cin, fullName);  
cout << "Your name is: " << fullName;
```

```
// Type your full name: John Doe  
// Your name is: John Doe
```

C++ Operators

C++ divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Assignment Operators

Operator	Example	Same As
=	$x = 5$	$x = 5$
+=	$x += 3$	$x = x + 3$
-=	$x -= 3$	$x = x - 3$
*=	$x *= 3$	$x = x * 3$
/=	$x /= 3$	$x = x / 3$

%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Comparison Operators

Comparison operators are used to compare two values (or variables). This is important in programming, because it helps us to find answers and make decisions.

The return value of a comparison is either **1** or **0**, which means **true** (1) or **false** (0). These values are known as **Boolean values**

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Logical Operators

As with , you can also test for **true** (1) or **false** (0) values with **logical operators**.

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
	Logical or	Returns true if one of the statements is true	x < 5 x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

C++ Math

C++ has many functions that allows you to perform mathematical tasks on numbers.

<cmath> Library

The **<cmath>** library has many functions that allow you to perform mathematical tasks on numbers.

Function	Description
<u>abs(x)</u>	Returns the absolute value of x
<u>acos(x)</u>	Returns the arccosine of x, in radians
<u>acosh(x)</u>	Returns the hyperbolic arccosine of x
<u>asin(x)</u>	Returns the arcsine of x, in radians
<u>asinh(x)</u>	Returns the hyperbolic arcsine of x
<u>atan(x)</u>	Returns the arctangent of x as a numeric value between -PI/2 and PI/2 radians
<u>atan2(y, x)</u>	Returns the angle theta from the conversion of rectangular coordinates (x, y) to polar coordinates (r, theta)
<u>atanh(x)</u>	Returns the hyperbolic arctangent of x
<u>cbrt(x)</u>	Returns the cube root of x
<u>ceil(x)</u>	Returns the value of x rounded up to its nearest integer
copysign(x, y)	Returns the first floating point x with the sign of the second floating point y
<u>cos(x)</u>	Returns the cosine of x (x is in radians)
<u>cosh(x)</u>	Returns the hyperbolic cosine of x
<u>exp(x)</u>	Returns the value of E^x
exp2(x)	Returns the value of 2^x
expm1(x)	Returns $e^x - 1$
erf(x)	Returns the value of the error function at x
erfc(x)	Returns the value of the complementary error function at x
<u>fabs(x)</u>	Returns the absolute value of a floating x
fdim(x)	Returns the positive difference between x and y

<u>floor(x)</u>	Returns the value of x rounded down to its nearest integer
<u>fma(x, y, z)</u>	Returns $x*y+z$ without losing precision
<u>fmax(x, y)</u>	Returns the highest value of a floating x and y
<u>fmin(x, y)</u>	Returns the lowest value of a floating x and y
<u>fmod(x, y)</u>	Returns the floating point remainder of x/y
frexp(x, y)	With x expressed as $m*2^n$, returns the value of m (a value between 0.5 and 1.0) and writes the value of n to the memory at the pointer y
<u>hypot(x, y)</u>	Returns $\sqrt{x^2 + y^2}$ without intermediate overflow or underflow
ilogb(x)	Returns the integer part of the floating-point base logarithm of x
ldexp(x, y)	Returns $x*2^y$
lgamma(x)	Returns the logarithm of the absolute value of the gamma function at x
llrint(x)	Rounds x to a nearby integer and returns the result as a long long integer
llround(x)	Rounds x to the nearest integer and returns the result as a long long integer
<u>log(x)</u>	Returns the natural logarithm of x
<u>log10(x)</u>	Returns the base 10 logarithm of x
log1p(x)	Returns the natural logarithm of $x+1$
<u>log2(x)</u>	Returns the base 2 logarithm of the absolute value of x
logb(x)	Returns the floating-point base logarithm of the absolute value of x
lrint(x)	Rounds x to a nearby integer and returns the result as a long integer
lround(x)	Rounds x to the nearest integer and returns the result as a long integer
modf(x, y)	Returns the decimal part of x and writes the integer part to the memory at the pointer y
nan(s)	Returns a NaN (Not a Number) value

<code>nearbyint(x)</code>	Returns x rounded to a nearby integer
<code>nextafter(x, y)</code>	Returns the closest floating point number to x in the direction of y
<code>nexttoward(x, y)</code>	Returns the closest floating point number to x in the direction of y
<code>pow(x, y)</code>	Returns the value of x to the power of y
<code>remainder(x, y)</code>	Return the remainder of x/y rounded to the nearest integer
<code>remquo(x, y, z)</code>	Calculates x/y rounded to the nearest integer, writes the result to the memory at the pointer z and returns the remainder.
<code>rint(x)</code>	Returns x rounded to a nearby integer
<code>round(x)</code>	Returns x rounded to the nearest integer
<code>scalbln(x, y)</code>	Returns $x \cdot R^y$ (R is usually 2)
<code>scalbn(x, y)</code>	Returns $x \cdot R^y$ (R is usually 2)
<code>sin(x)</code>	Returns the sine of x (x is in radians)
<code>sinh(x)</code>	Returns the hyperbolic sine of x
<code>sqrt(x)</code>	Returns the square root of x
<code>tan(x)</code>	Returns the tangent of x (x is in radians)
<code>tanh(x)</code>	Returns the hyperbolic tangent of x
<code>tgamma(x)</code>	Returns the value of the gamma function at x
<code>trunc(x)</code>	Returns the integer part of x

C++ Structures

Structures (also called structs) are a way to group several related variables into one place. Each variable in the structure is known as a **member** of the structure.

To create a structure, use the **struct** keyword and declare each of its members inside curly braces.

```
struct {           // Structure declaration
    int myNum;     // Member (int variable)
```

```
string myString; // Member (string variable)
} myStructure;   // Structure variable
```

Access Structure Members

To access members of a structure, use the dot syntax (.)

```
#include <iostream>

#include <string>

using namespace std;

int main() {

    // Create a structure variable called myStructure
    struct {
        int myNum;
        string myString;
    } myStructure;

    // Assign values to members of myStructure
    myStructure.myNum = 1;
    myStructure.myString = "Hello World!";

    // Print members of myStructure
    cout << myStructure.myNum << "\n";
    cout << myStructure.myString << "\n";

}
```

One Structure in Multiple Variables

You can use a comma (,) to use one structure in many variables:

```
struct {
    int myNum;
    string myString;
} myStruct1, myStruct2, myStruct3; // Multiple structure variables separated with commas
```

Named Structures

By giving a name to the structure, you can treat it as a data type. This means that you can create variables with this structure anywhere in the program at any time.

To create a named structure, put the name of the structure right after the **struct** keyword:

```
struct myDataType { // This structure is named "myDataType"
    int myNum;
    string myString;
};
```

o declare a variable that uses the structure, use the name of the structure as the data type of the variable:

```
myDataType myVar;
```

note – by default structure members are public

Union

Advantages of Structure

- o Structure stores more than one data type of the same object together.
 - o It is helpful when you want to store data of different or similar data types such as name, address, phone, etc., of the same object.
 - o It makes it very easy to maintain the entire record as we represent complete records using a single name.
 - o The structure allows passing a whole set of records to any function with the help of a single parameter.
 - o An array of structures can also be created to store multiple data of similar data types.
-

Disadvantages of Structure

- o If the complexity of the project goes increases, it becomes hard to manage all your data members.
 - o Making changes to one data structure in a program makes it necessary to change at several other places. So it becomes difficult to track all changes.
 - o The structure requires more storage space as it allocates memory to all the data members, and even it is slower.
 - o The structure takes more storage space as it gives memory to all the different data members, whereas union takes only the memory size required by the largest data size parameters, and the same memory is shared with other data members.
-

Comparative advantages and disadvantages of union

Below is the list of some advantages and disadvantages of using union:

Advantages of Union

- o Union takes less memory space as compared to the structure.
 - o Only the largest size data member can be directly accessed while using a union.
 - o It is used when you want to use less (same) memory for different data members.
 - o It allocates memory size to all its data members to the size of its largest data member.
-

Disadvantages of Union

- o It allows access to only one data member at a time.
 - o Union allocates one single common memory space to all its data members, which are shared between all of them.
 - o Not all the union data members are initialized, and they are used by interchanging values at a time.
-

C++ Enums

An **enum** is a special type that represents a group of constants (unchangeable values).

To create an enum, use the **enum** keyword, followed by the name of the enum, and separate the enum items with a comma:

```
enum Level {  
    LOW,  
    MEDIUM,  
    HIGH  
};
```

Note that the last item does not need a comma.

To access the enum, you must create a variable of it.

```
#include <iostream>  
  
using namespace std;  
  
enum Level {  
  
    LOW,  
  
    MEDIUM,  
  
    HIGH  
  
};  
  
int main() {  
  
    enum Level myVar = MEDIUM;  
  
    cout << myVar;  
  
    return 0;  
  
}
```

C++ Pointers

A variable that holds a memory address is called Pointer.

Syntax – datatype * Pointer

A pointer variable points to a data type (like `int` or `string`) of the same type, and is created with the `*` operator. The address of the variable you're working with is assigned to the pointer:

```
#include <iostream>

#include <string>

using namespace std;

int main() {

    string food = "Pizza"; // A string variable

    string* ptr = &food; // A pointer variable that stores the address of food

    // Output the value of food

    cout << food << "\n";

    // Output the memory address of food

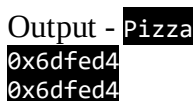
    cout << &food << "\n";

    // Output the memory address of food with the pointer

    cout << ptr << "\n";

    return 0;

}
```



Output - Pizza
0x6dfed4
0x6dfed4

Get Memory Address and Value

In the above example, we used the pointer variable to get the memory address of a variable (used together with the `&` **reference** operator). However, you can also use the pointer to get the value of the variable, by using the `*` operator (the **dereference** operator):

```
#include <iostream>

#include <string>
```

```

using namespace std;

int main() {

    string food = "Pizza"; // Variable declaration

    string* ptr = &food;   // Pointer declaration

    // Reference: Output the memory address of food with the pointer

    cout << ptr << "\n";

    // Dereference: Output the value of food with the pointer

    cout << *ptr << "\n";

    return 0;
}

```

O/P - 0x6dfed4
Pizza

Modify the Pointer Value

You can also change the pointer's value. But note that this will also change the value of the original variable:

```

#include <iostream>

#include <string>

using namespace std;

int main() {

    string food = "Pizza";

    string* ptr = &food;

    // Output the value of food

    cout << food << "\n";

    // Output the memory address of food

    cout << &food << "\n";

    // Access the memory address of food and output its value

    cout << *ptr << "\n";
}

```

```
// Change the value of the pointer

*ptr = "Hamburger";

// Output the new value of the pointer

cout << *ptr << "\n";

// Output the new value of the food variable

cout << food << "\n";

return 0;
```

```
}O/P - Pizza
0x6dfed4
Pizza
Hamburger
Hamburger
```

C++ What is OOP?

OOP stands for Object-Oriented Programming.

C++ Classes/Objects

C++ is an object-oriented programming language.

Everything in C++ is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an **object**. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.

Attributes and methods are basically **variables** and **functions** that belongs to the class. These are often referred to as "class members".

A class is a user-defined data type that we can use in our program, and it works as an object constructor, or a "blueprint" for creating objects.

Creating a class -

Example

Create a class called "MyClass":

```
class MyClass {           // The class
public:                   // Access specifier
    int myNum;            // Attribute (int variable)
    string myString;      // Attribute (string variable)
};
```

Note - Class members are by default private

Create an Object

In C++, an object is created from a class. We have already created the class named `MyClass`, so now we can use this to create objects.

To create an object of `MyClass`, specify the class name, followed by the object name.

To access the class attributes (`myNum` and `myString`), use the dot syntax (`.`) on the object:

```
#include <iostream>

#include <string>

using namespace std;

class MyClass {    // The class

    public:        // Access specifier

        int myNum;    // Attribute (int variable)

        string myString; // Attribute (string variable)

};

int main() {

    MyClass myObj; // Create an object of MyClass

    // Access attributes and set values

    myObj.myNum = 15;

    myObj.myString = "Some text";

    // Print values

    cout << myObj.myNum << "\n";

    cout << myObj.myString;

    return 0;

}
```

Multiple Objects

You can create multiple objects of one class:

Class Methods

Methods are **functions** that belongs to the class.

There are two ways to define functions that belongs to a class:

- Inside class definition
- Outside class definition

In the following example, we define a function inside the class, and we name it "myMethod".

Note: You access methods just like you access attributes; by creating an object of the class and using the dot syntax (.):

```
#include <iostream>

using namespace std;

class MyClass {    // The class

public:            // Access specifier

    void myMethod() { // Method/function

        cout << "Hello World!";

    }

};

int main() {

    MyClass myObj;    // Create an object of MyClass

    myObj.myMethod(); // Call the method

    return 0;

}
```

To define a function outside the class definition, you have to declare it inside the class and then define it outside of the class. This is done by specifying the name of the class, followed the scope resolution :: operator, followed by the name of the function:

```
#include <iostream>

using namespace std;

class MyClass {    // The class

public:            // Access specifier
```

```
void myMethod(); // Method/function declaration

};

// Method/function definition outside the class

void MyClass::myMethod() {

    cout << "Hello World!";

}

int main() {

    MyClass myObj; // Create an object of MyClass

    myObj.myMethod(); // Call the method

    return 0;

}
```